



UNIVERSITY OF
MICHIGAN

EECS 453

Principles of

Machine Learning

Lecture 2 – Basics of Supervised Learning

Instructor: Prof. Qing Qu

Lecture Agenda

- **Basics Mathematics**
- Basic Concepts of Supervised Learning
- Basic Example: Linear Regression
- Fundamental Trade-offs in Supervised Learning

Basic Notions of Linear Algebra

- **Basic notation:** we use x to denote a scalar, boldface $\mathbf{x}$ for a vector, and boldface $\mathbf{X}$ for a matrix.
- **Vectors:** $\mathbf{x} \in \mathbb{R}^n$ is a real-valued n -dimensional column vector; i.e.,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad x_i \in \mathbb{R}, \quad \forall i.$$

Basic Notions of Linear Algebra

- **Transpose:** let $\mathbf{x} \in \mathbb{R}^n$. The notation $\mathbf{x}^\top$ means that

$$\mathbf{x}^\top = [x_1 \quad x_2 \quad \cdots \quad x_n].$$

- **Matrix:** $\mathbf{A} \in \mathbb{R}^{m \times n}$ means that $\mathbf{A}$ is a real-valued $m \times n$ matrix that

$$\mathbf{A} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}, \quad a_{ij} \in \mathbb{R}, \quad \forall i, j.$$

Basic Notions of Linear Algebra

- **Inner product.** Given two vectors $\mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \mathbb{R}^n$, their inner product is defined as

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i$$

- **Norms.** For a vector $\mathbf{x} = [x_1 \ x_2 \ \cdots \ x_n]^\top \in \mathbb{R}^n$,

$$\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}} = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle} = \sqrt{\sum_{i=1}^n x_i^2},$$

which measures the length of $\mathbf{x}$.

Expectation of Random Variables

- **Expectation.** Suppose X is a random variable ($r.v.$), its expectation is denoted as $\mathbb{E}[X]$.
- **Discrete $r.v.$** Suppose X take discrete values $x_1, \dots, x_k$ with probability $p_1, \dots, p_k$, then

$$\mathbb{E}[X] = \sum_{i=1}^k p_i x_i$$

- **Continuous $r.v.$** Suppose X takes continuous values in $(-\infty, \infty)$ with density $p(x)$, then

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} p(x)x dx$$

Basic Notions of Optimization

- **Optimization problems:** find the minimum value of a function $f : \mathbb{R}^n \mapsto \mathbb{R}$ via

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$$

- **Optimal solution:** find the point $\mathbf{x}_\star$ (called the optimal solution) that achieves the minimum value of the function $f(\cdot)$ over $\mathbb{R}^n$, i.e.,

$$f(\mathbf{x}_\star) = \min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}).$$

Clearly, $\mathbf{x}_\star = \arg \min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x})$.

Linear Algebra & Probability Review

- Review of Linear Algebra: 08/28, Friday
- Review of Probability: 09/04, Friday
- Review of Python: 09/11, Friday
- Instructor: Huijie Zhang, huijiezhang@umich.edu
- Materials & Slides: available on Canvas

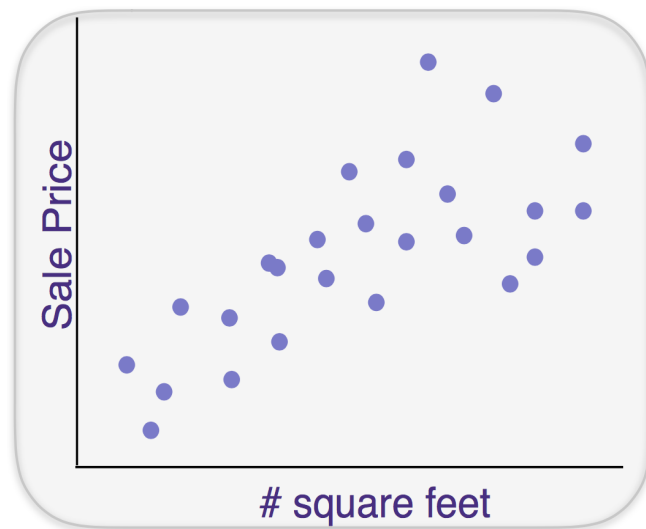
Lecture Agenda

- Basics Mathematics
- **Basic Concepts of Supervised Learning**
- Basic Example: Linear Regression
- Fundamental Trade-offs in Supervised Learning

Supervised Learning

Supposed we have a dataset giving the living areas and prices of 47 houses from Portland, Oregon:

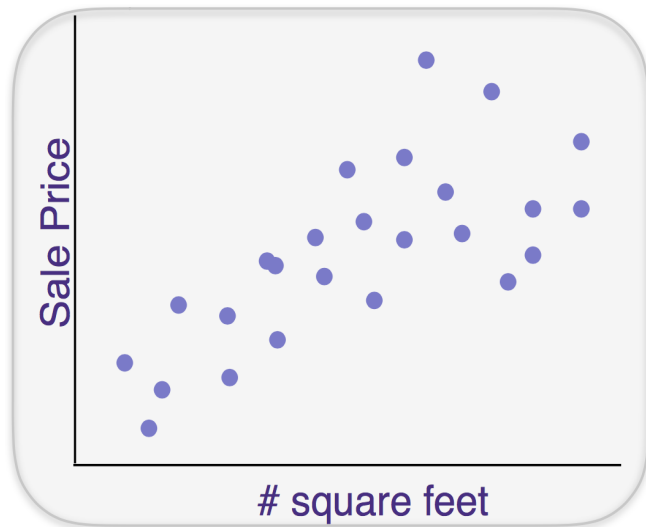
Living area (sqft)	Price (1000\$s)
2104	400
1600	330
2400	369
1416	232
3000	540
...	...



Supervised Learning

Question: how can we learn from given data to predict the prices of other houses in Portland based on living area?

Living area (sqft)	Price (1000\$s)
2104	400
1600	330
2400	369
1416	232
3000	540
...	...



Supervised Learning

To establish notation for the future use:

- $x^{(i)}$: the **input** variable (aka living area)
- $y^{(i)}$: the **output** prediction (aka housing price)
- $(x^{(i)}, y^{(i)})$: one (e.g., i -th) training example
- $\{(x^{(i)}, y^{(i)}); i = 1, \dots, n\}$: **training set**, a list of training examples

***Note:** the superscript “ (i) ” in the notation is simply an index into the training set, and has nothing to do with exponentiation.*

Supervised Learning

To establish notation for the future use:

- $\mathcal{X}$: the space of input values
- $\mathcal{Y}$: the space of output values

In our vanilla housing prediction example, $\mathcal{X} = \mathcal{Y} = \mathbb{R}$.

Goal: given a training set $\{(x^{(i)}, y^{(i)}); i = 1, \dots, n\}$,
learn a function

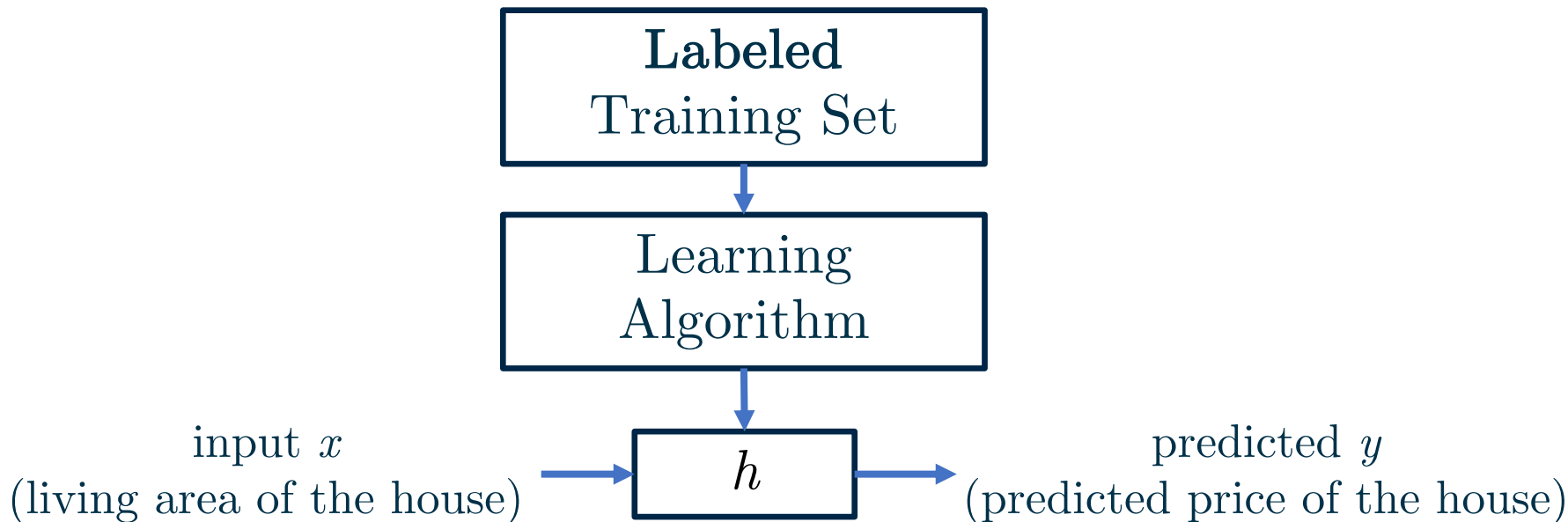
$$h : \mathcal{X} \rightarrow \mathcal{Y}$$

so that $h(x)$ is a “good” predictor for y .

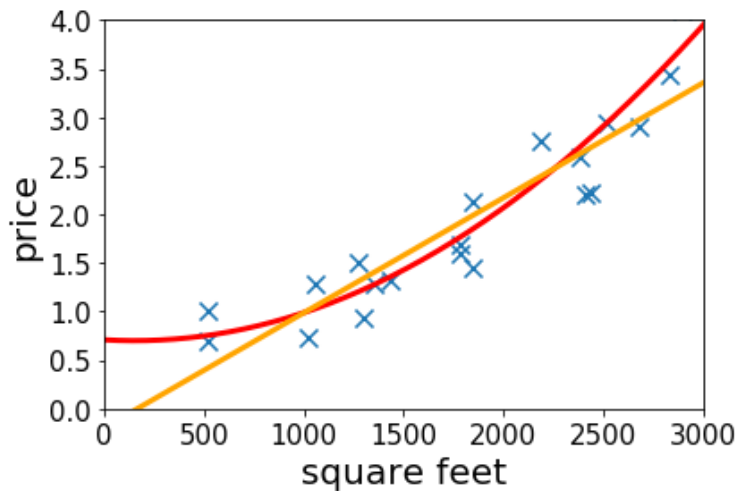
For historical reasons, this function h is called a **hypothesis**.

Supervised Learning

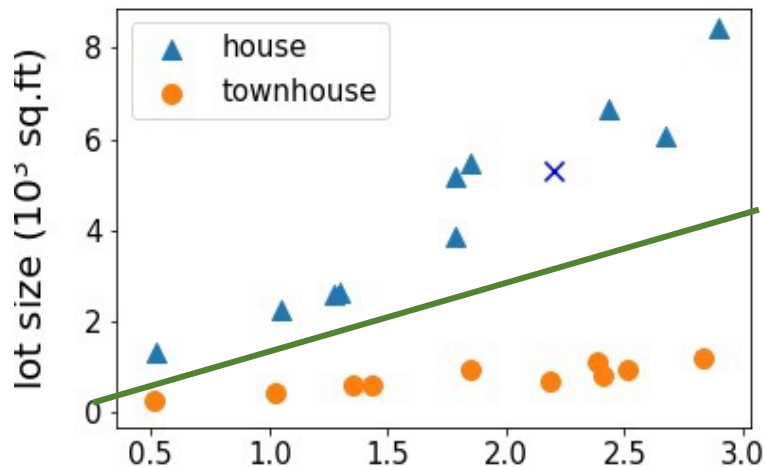
Seen pictorially, the process is therefore like the following



Supervised Learning



Regression: the prediction y is *continuous*



Classification: the prediction y is *discrete*

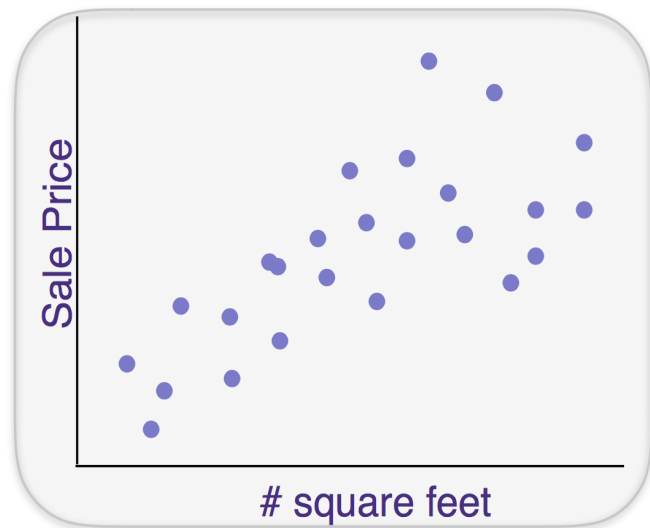
Lecture Agenda

- Basics Mathematics
- Basic Concepts of Supervised Learning
- **Basic Example: Linear Regression**
 - **From One-dim to High-dim Linear Regression**
 - Solution of Least Squares and Interpretations
- Fundamental Trade-offs in Supervised Learning

1-dimensional Regression Problem

Example: House price prediction (again)

- **Training Data:** given past sales data $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$, where $x^{(i)} \in \mathbb{R}$ represents square feet, and $y^{(i)}$ represents the sale price.
- **Task:** given a new house with square meter x , predict its sales price y



1-dimensional Regression Problem

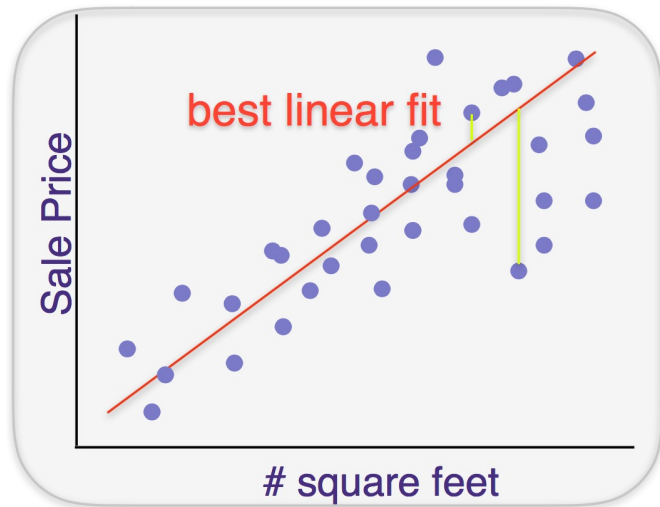
- **Decide hypothesis/models:**

the house price is a linear function of the square feet

$$y^{(i)} \approx \theta x^{(i)}$$

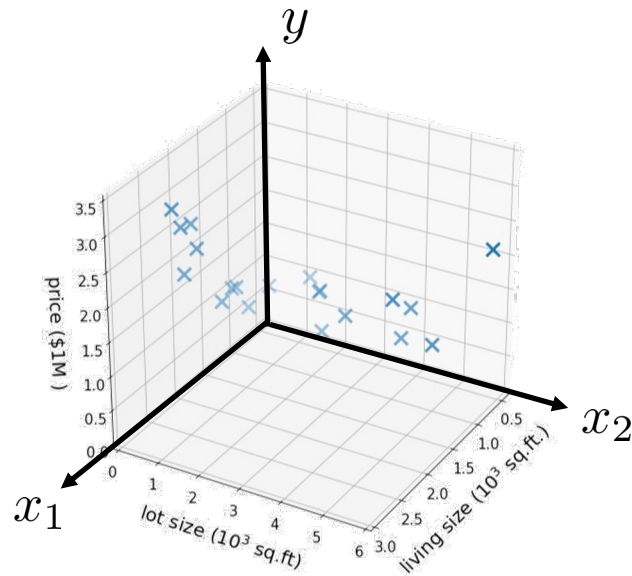
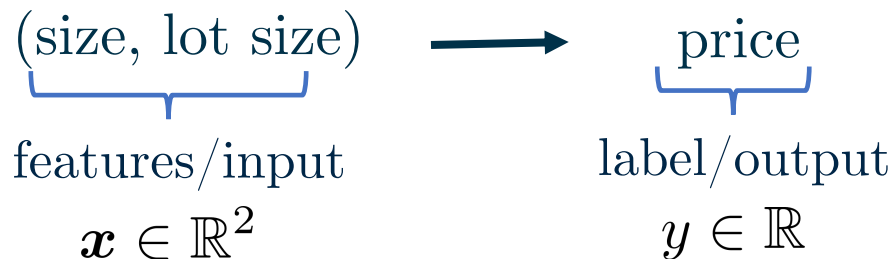
- **Learning problem:** design a learning algorithm to fit the data best

$$\theta_{\star} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \theta x^{(i)})^2$$



High-dimensional Regression Problem

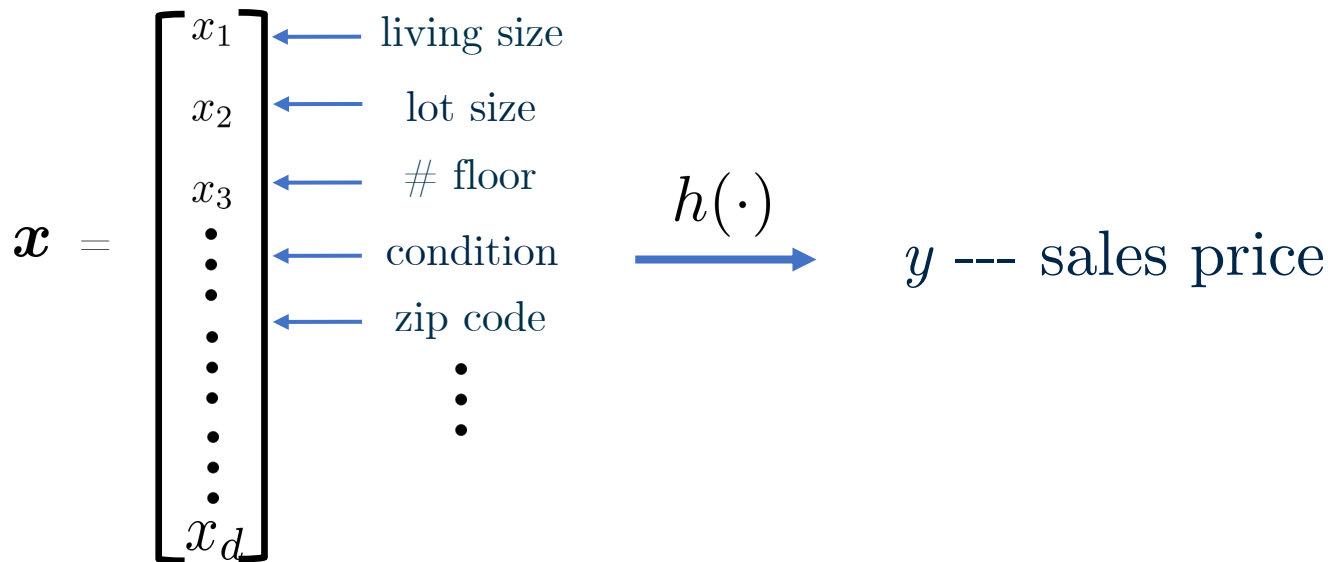
- Suppose we also know the **lot size**
- **Task:** find a function that maps



- **Dataset:** $(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$, where $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)})$
- “Supervision” refers to the labels $y^{(1)}, \dots, y^{(n)}$

High-dimensional Regression Problem

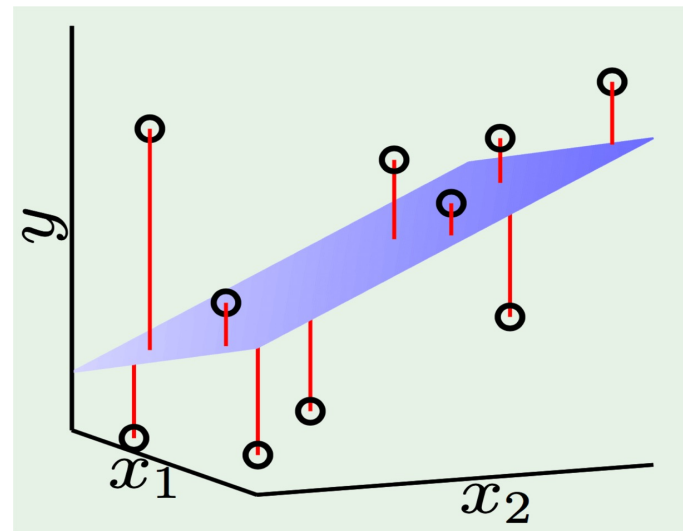
- Higher dimension of the input: $\mathbf{x} \in \mathbb{R}^d$ for a large d
- E.g.,



High-dimensional Regression Problem

- **Data:** given past sales data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ represents living size, lot size, # floors, conditions, schools ...
 $y^{(i)}$ represents sales price;
- **Hypothesis/model:**
 $y^{(i)} \approx \mathbf{x}^{(i)\top} \boldsymbol{\theta}$, where $\boldsymbol{\theta} \in \mathbb{R}^d$
- **Learning problem:** solve a *least squares* (LS) problem

$$\boldsymbol{\theta}_\star = \arg \min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \mathbf{x}^{(i)\top} \boldsymbol{\theta})^2$$



Lecture Agenda

- Basics Mathematics
- Basic Concepts of Supervised Learning
- **Basic Example: Linear Regression**
 - From 1-dim to High-dim Linear Regression
 - **Solution of Least Squares and Interpretations**
- Fundamental Trade-offs in Supervised Learning

Least Squares in Matrix Forms

- Matrix representation:

$$\begin{aligned} \mathbf{X} &= \begin{bmatrix} \mathbf{x}^{(1)\top} \\ \vdots \\ \mathbf{x}^{(n)\top} \end{bmatrix} \in \mathbb{R}^{n \times d} \\ \mathbf{y} &= \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} \in \mathbb{R}^n \end{aligned} \quad \longrightarrow \quad \begin{bmatrix} y^{(1)} - \mathbf{x}^{(1)\top} \boldsymbol{\theta} \\ \vdots \\ y^{(n)} - \mathbf{x}^{(n)\top} \boldsymbol{\theta} \end{bmatrix} = \mathbf{y} - \mathbf{X}\boldsymbol{\theta}$$

- Least squares:

$$\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \mathbf{x}^{(i)\top} \boldsymbol{\theta} \right)^2 \iff \boxed{\min_{\boldsymbol{\theta}} \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2}$$

Gradients of Functions

- **Definition:** the gradient of a *continuously differentiable* function $f : \mathbb{R}^d \mapsto \mathbb{R}$ at $\boldsymbol{\theta}$ is given by

$$\nabla f(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \vdots \\ \frac{\partial f}{\partial \theta_d} \end{bmatrix} \in \mathbb{R}^d$$

which always has the same dimension as $\boldsymbol{\theta} \in \mathbb{R}^d$.

- If $f(\boldsymbol{\theta}) = \boldsymbol{\theta}^\top \boldsymbol{x}$, then $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) = \boldsymbol{x}$.
- If $f(\boldsymbol{\theta}) = \boldsymbol{\theta}^\top \boldsymbol{X} \boldsymbol{\theta}$ and $\boldsymbol{X}$ is symmetric, then $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) = 2\boldsymbol{X}\boldsymbol{\theta}$.

Solution of Least Squares

- **Objective:**

$$\begin{aligned} L(\boldsymbol{\theta}) &= \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 = (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) \\ &= \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y} \end{aligned}$$

- **Take the gradient:**

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = 2\mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

- **Set the gradient to zero:** the optimal solution $\boldsymbol{\theta}_\star$ satisfies

$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta}_\star = \mathbf{X}^\top \mathbf{y}$$

- **Question:** how to solve for $\boldsymbol{\theta}_\star$ based on the above linear equations?

Solution of Least Squares

Case I: $\mathbf{X} \in \mathbb{R}^{n \times d}$ has **full** column rank ($n \geq d$)

- $\mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{d \times d}$ is *non-singular* (i.e., invertible)
- The optimal solution has a *closed-form*

$$\boldsymbol{\theta}_\star = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^\dagger \mathbf{y}$$

where

$$\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \in \mathbb{R}^{d \times n}$$

denotes the **pseudo-inverse** of the matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, whenever it has full column rank.

Note: We have $\mathbf{X}^\dagger \mathbf{X} = \mathbf{I}$, but not necessarily $\mathbf{X} \mathbf{X}^\dagger = \mathbf{I}$.

Geometric Interpretation of LS Solutions

$$\theta_{\star} = \left(\mathbf{X}^{\top} \mathbf{X} \right)^{-1} \mathbf{X}^{\top} \mathbf{y} = \mathbf{X}^{\dagger} \mathbf{y}$$

- Projection operator:

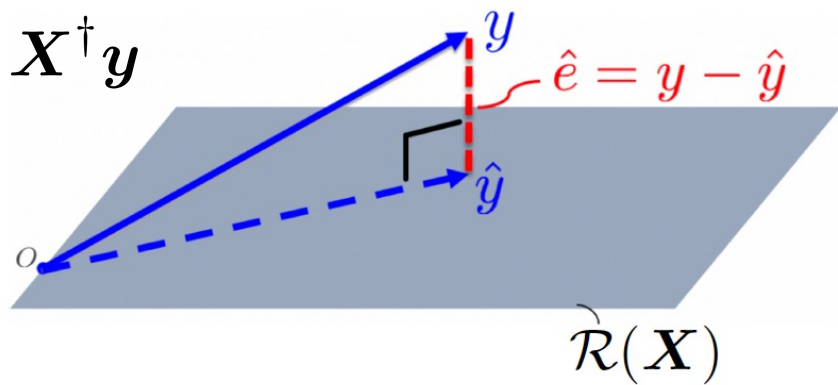
$$\mathcal{P}_{\mathbf{X}} = \mathbf{X} \mathbf{X}^{\dagger}$$

onto the range space $\mathcal{R}(\mathbf{X})$

$$\mathcal{R}(\mathbf{X}) = \text{span} \left\{ \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(d)} \right\}$$

The LS solution is such that $\hat{\mathbf{y}} = \mathbf{X} \theta_{\star} = \mathcal{P}_{\mathbf{X}}(\mathbf{y})$ is the **orthogonal projection** onto the range space $\mathcal{R}(\mathbf{X})$:

$$\mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X} \theta_{\star} = \mathbf{y} - \mathbf{X} \mathbf{X}^{\dagger} \mathbf{y} = \mathbf{y} - \mathcal{P}_{\mathbf{X}}(\mathbf{y})$$



Solution of Least Squares

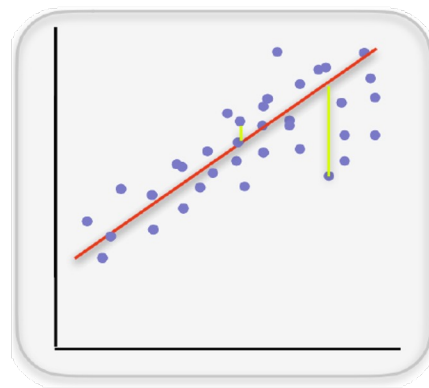
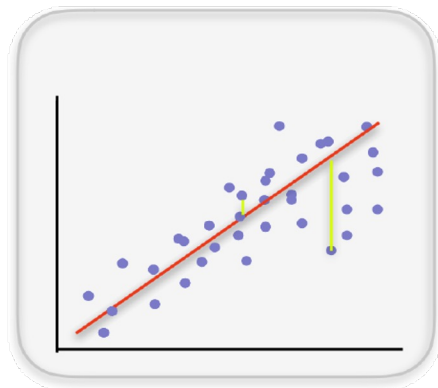
Case II: $\mathbf{X} \in \mathbb{R}^{n \times d}$ does NOT have full column rank

- $\boldsymbol{\theta}$ does not have unique solutions
- Typically, $n < d$, thus
 - “**overfitting**”: more model parameters than can be justified by the number of data points
- How to obtain unique and meaningful solution?
 - “**regularization**”: e.g., $\min_{\boldsymbol{\theta}} \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2$

Will discuss later in this lecture...

Dealing with the Offset (Bias)

Currently:



How to be more general?

Adding a bias $y_i \approx \mathbf{x}_i^\top \boldsymbol{\theta} + b$, $i = 1, \dots, n$ where $b \in \mathbb{R}$

Dealing with the Offset (Bias)

- New least squares formulation:

$$\min_{\boldsymbol{\theta}, b} \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} + \mathbf{1}b - \mathbf{y}\|_2^2.$$

- Rewrite the problem as

$$\min_{\tilde{\boldsymbol{\theta}}} \frac{1}{n} \|\widetilde{\mathbf{X}}\tilde{\boldsymbol{\theta}} - \mathbf{y}\|_2^2,$$

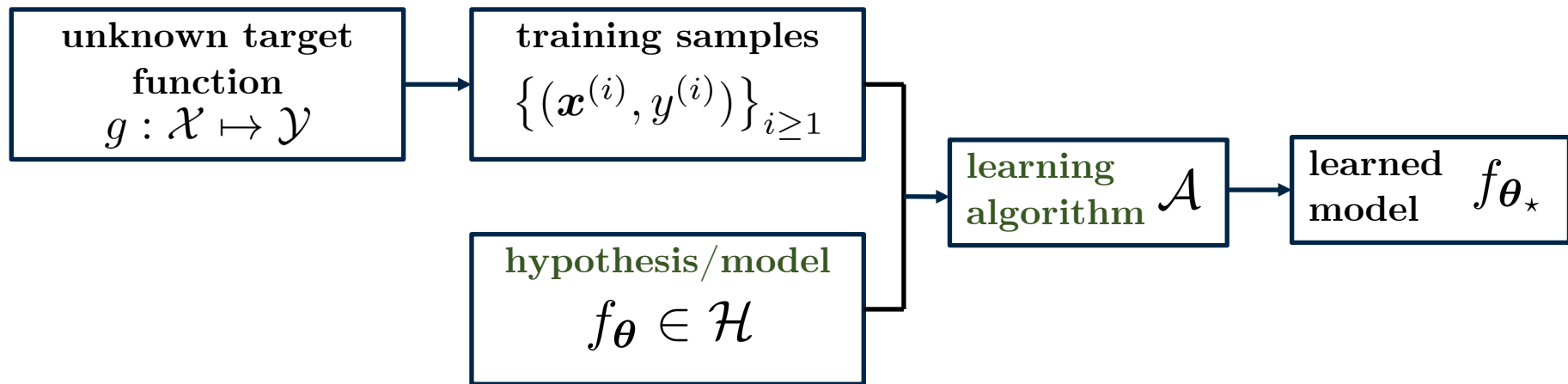
$$\widetilde{\mathbf{X}} = \begin{bmatrix} \mathbf{x}_1^\top & 1 \\ \vdots & \\ \mathbf{x}_n^\top & 1 \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}, \quad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ b \end{bmatrix} \in \mathbb{R}^{d+1}.$$

No fundamental difference.

Lecture Agenda

- Basics Mathematics
- Basic Concepts of Supervised Learning
- Basic Example: Linear Regression
- **Fundamental Trade-offs in Supervised Learning**

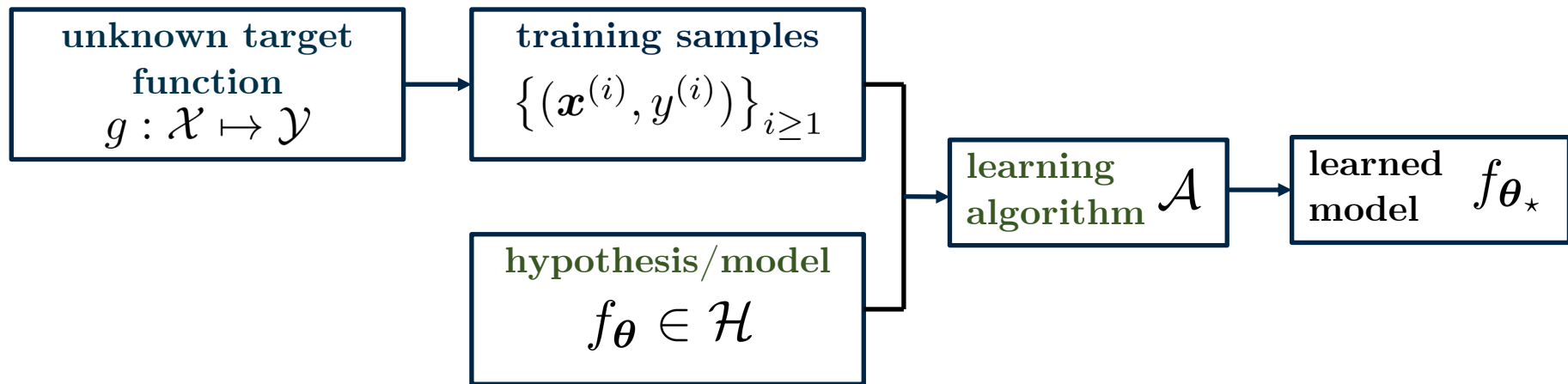
Supervised Learning: High-level Review



Linear regression example:

- Hypothesis/model: linear model $\mathbf{y} = \mathbf{X}\boldsymbol{\theta}$, model parameter $\boldsymbol{\theta}$;
- Learning method: solving the least squares problem;
- Learned model: $\boldsymbol{\theta}_* = \arg \min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$.

Supervised Learning: High-level Review



- We desire:

$$f_{\theta_*} \approx g$$

- **Prediction:** a new sample data $\mathbf{x}$ with unknown label, predict

$$\hat{y} \leftarrow f_{\theta_*}(\mathbf{x})$$

Measure of Learning Error

- The learning goal: $f_{\theta_*} \approx g$
- Measure of the learning error:
 - Almost always point-wise error measure:
 $e(f(\mathbf{x}), g(\mathbf{x}))$ is small for all possible data $\mathbf{x}$
 - Examples:

squared error: $e(f(\mathbf{x}), g(\mathbf{x})) = (f(\mathbf{x}) - g(\mathbf{x}))^2$

binary error: $e(f(\mathbf{x}), g(\mathbf{x})) = \mathbb{1}_{f(\mathbf{x}) \neq g(\mathbf{x})}$

Measure of Learning Risk

- The learning goal: $f_{\theta_*} \approx g$

- Measure of learning risk:

- Given a set of training samples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$

$$\text{empirical risk } R_e = \frac{1}{n} \sum_{i=1}^n e(f(\mathbf{x}^{(i)}), g(\mathbf{x}^{(i)}))$$

- For regression problems, if $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$ is the set of all training data, then $R_e = \text{Er}_{train}$;
- Suppose data $\mathbf{x}$ follows a certain distribution $\mathbf{x} \sim \mathcal{D}$

$$\text{population risk } R_p = \mathbb{E}_{\mathbf{x}} [e(f(\mathbf{x}), g(\mathbf{x}))]$$

Measure of Learning Risk

- The learning goal: $f_{\theta_*} \approx g$
- Measure of learning risk:

➤ Empirical risk:

$$\text{empirical risk } R_e = \frac{1}{n} \sum_{i=1}^n e(f(\mathbf{x}^{(i)}), g(\mathbf{x}^{(i)}))$$

is also called the *in-sample error*.

➤ Population risk:

$$\text{population risk } R_p = \mathbb{E}_{\mathbf{x}} [e(f(\mathbf{x}), g(\mathbf{x}))]$$

is also called the *out-of-sample error*.

The Goal of Supervised Learning

make the population risk/test error small

- **Analogy to taking midterm exams**
 - The empirical risk/training error is the practice exam;
 - The population risk/test error is the actual exam;
 - Goal: do well on the actual exam;



(Training stage)



(Testing stage)

The Goal of Supervised Learning

make the population risk/test error small

- **Analogy to taking midterm exams**
 - The empirical risk/training error is the practice exam;
 - The population risk/test error is the actual exam;
 - Goal: do well on the actual exam;
- **Memorization vs learning**
 - can do well on training data by memorizing it using more complex models (**overfitting**)
 - learning means that you have to do well with new/unseen data (**generalization**)

The Rule of Supervised Learning

Rule: The test data can NEVER influence the training phase in any way.

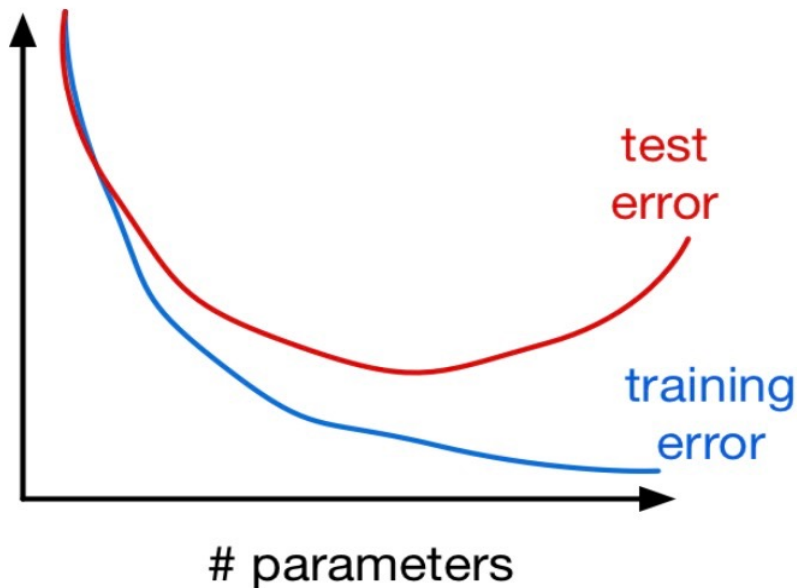
- **i.i.d. assumption** (ideal):
 - All data (training + testing) come from the same distribution
 - The data are sampled independently
- **Learning theory:**

Explore how population risk/test error is related to empirical risk/training error

Q: Does empirical risk small imply population risk small?

The Rule of Supervised Learning

Q: *Does empirical risk/training error small imply population risk/test error small?*

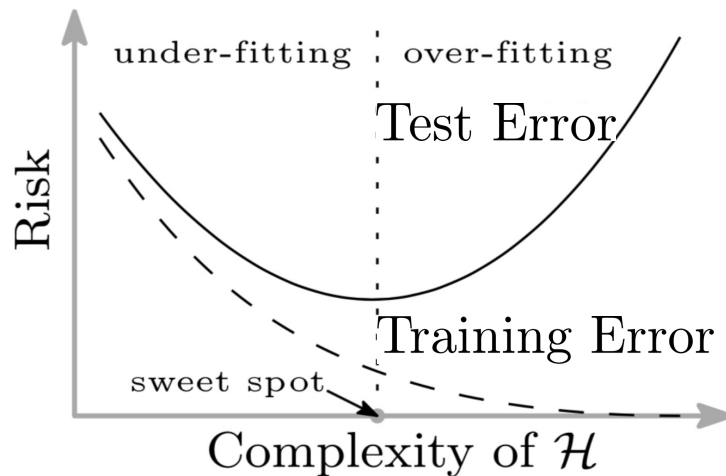


The Fundamental Trade-off in Learning

$$R_p = \underbrace{R_p - R_e}_{\text{generalization}} + \underbrace{R_e}_{\text{training}}$$

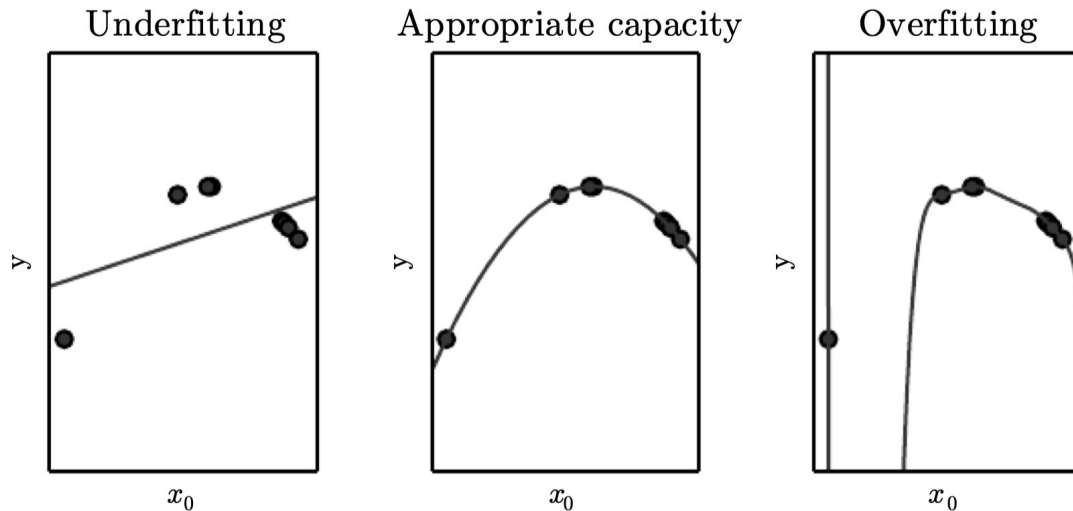
- On the **training** side, we need:
more complex model/hypothesis $\mathcal{H}$
- On the **generalization** side, we need:
less complex model/hypothesis $\mathcal{H}$
- Machine learning is *fundamentally* about **generalization**
Bounding the generalization error

The Fundamental Trade-off in Learning



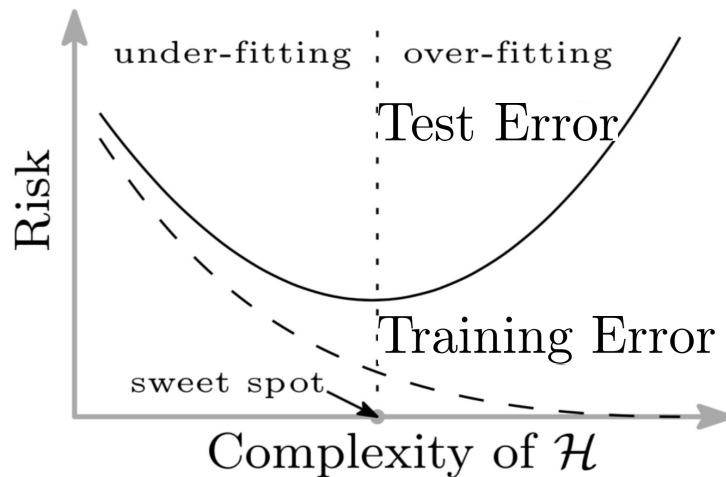
- **Underfitting:** *poor* performance on the training data, and *poor* generalization to the test data
- **Overfitting:** *good* performance on the training data, *poor* generalization to the test data

The Fundamental Trade-off in Learning



- **Underfitting:** *poor* performance on the training data, and *poor* generalization to the test data
- **Overfitting:** *good* performance on the training data, *poor* generalization to the test data

The Fundamental Trade-off in Learning



Q: underfitting vs. overfitting, which one is worse?

A: Overfitting, as the test error can be unbounded. Also, larger models nowadays are more prevalent (e.g., deep network) with tendency to overfit

Prevent Overfitting

Q: How to find the sweet spot?

- **Natural idea:** adjust the model size properly, but how?
- **Using validation set:** use part of the training data as **validation set** to approximate the test error

Split training examples into **training set** and **validation set**:

- Train model based on the training data
- Test the model based on the validation data

Prevent Overfitting

Q: How to find the sweet spot?

- **Natural idea:** adjust the model size properly, but how?
- **Using validation set:** use part of the training data as **validation set** to approximate the test error
 - **Training set:** a set of training examples the model trained on
 - **Validation set:** used to tune hyperparameters of our models (e.g., model size, regularizations, learning rate)
 - **Test set:** which is used to measure the generalization performance

The losses on these subsets are called **training, validation, and test losses**, respectively.

Prevent Overfitting

Q: How to find the sweet spot?

- **Natural idea:** adjust the model size properly, but how?
- **Using validation set:** use part of the training data as **validation set** to approximate the test error
 - We **cannot tune hyperparameters on the training set**, because we want to choose values that will generalize.
 - We also **cannot tune them on the test set**, because that would be “**cheating.**” We’re only allowed to use the test set once, to report the final performance.
 - Under i.i.d. assumption, $\mathbb{E}[R_{val}] = \mathbb{E}[R_p]$

Prevent Overfitting I

- (k-fold) Cross Validation $n = 8$



Example:

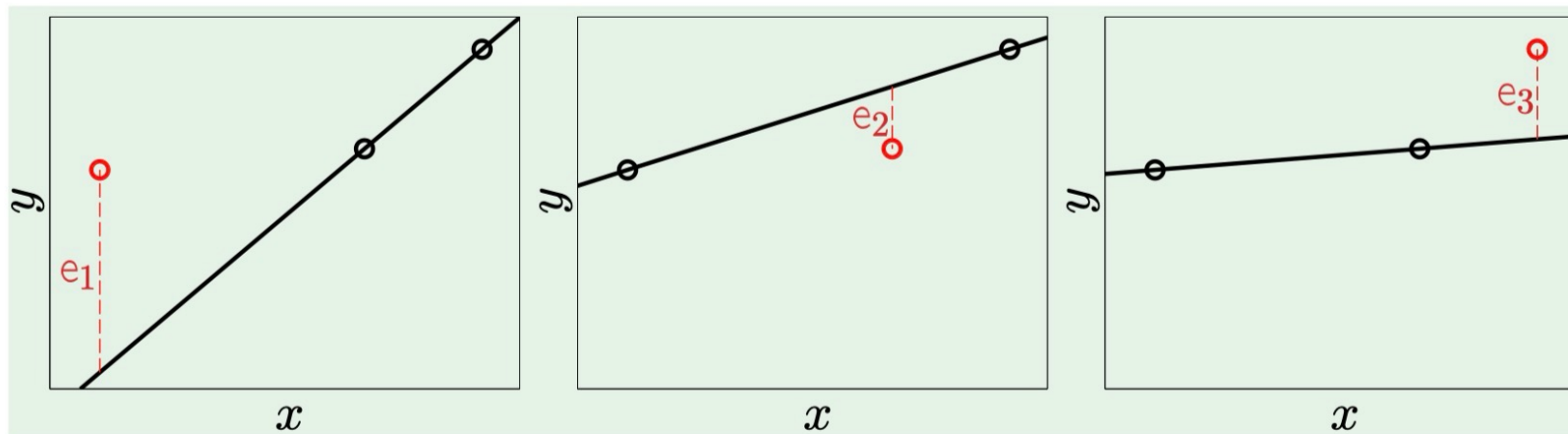
- Train Model 1, compute validation error
- ...
- Train Model 8, compute validation error
- Return the **average** validation error

Model 1



Fine tune the hyperparameters based on the **average** validation error.

Prevent Overfitting I



$$\text{Er}_{\text{cv}} = \frac{1}{3}(e_1 + e_2 + e_3)$$

Prevent Overfitting II: Regularization

Q: How to find the sweet spot?

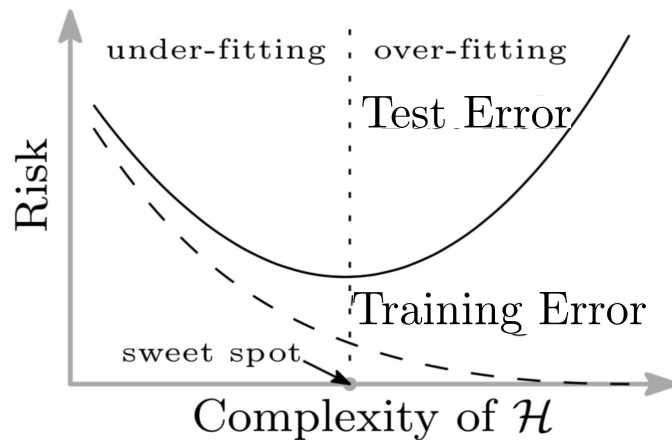
- **Answer:** handicapping the minimization of Er_{in} via regularization

- **L2-regularization** (weight decay regularization)

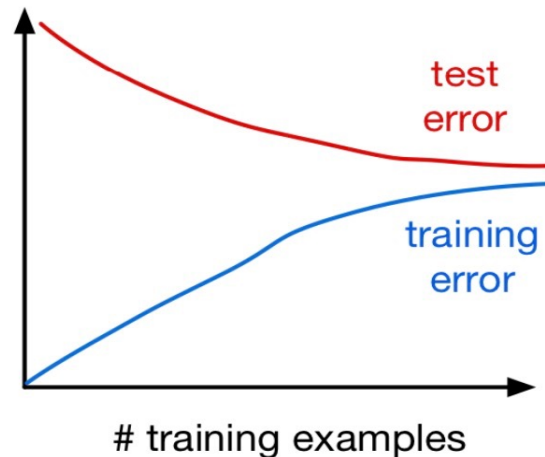
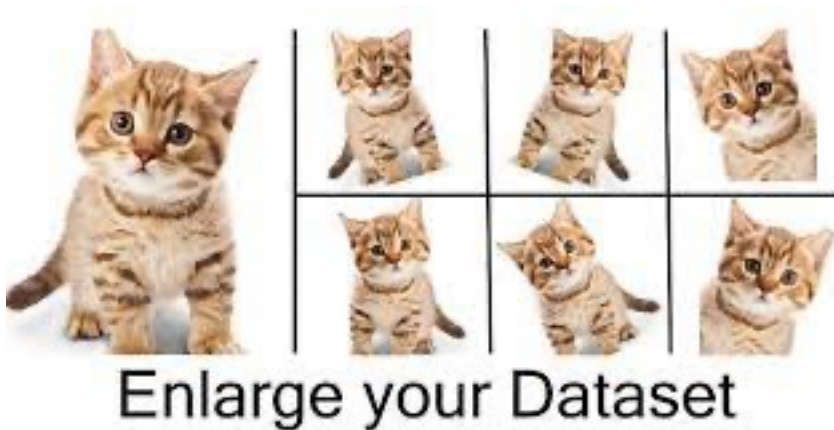
$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} f_{\boldsymbol{\theta}}(\boldsymbol{x}, y) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2$$

- **L1-regularization**

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} f_{\boldsymbol{\theta}}(\boldsymbol{x}, y) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_1$$

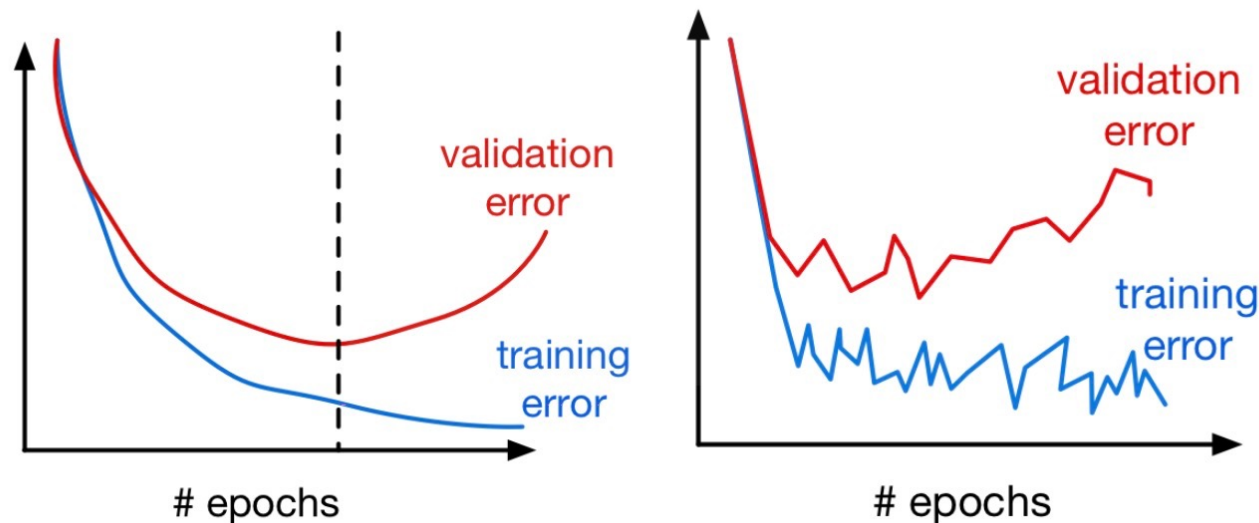


Prevent Overfitting III: Data Augmentation



- **Data augmentation:** artificially augment the training set by introducing distortions into the inputs (commonly used in vision tasks)
- **Approaches:** random rotation, adding noise, shifting, warping, etc.
- **Intuition:** Increase the effective size of the training set, and make it more likely that any given test example has a closely related training example

Prevent Overfitting IV: Early Stopping



- Stop the training at the point where the generalization error starts to increase (on validation set)

Further Readings & References

- [Deep Learning](#), by Ian Goodfellow, Yoshua Bengio, and Aaron Courville. (Chapter 2&3, Chapter 5)
- [The Elements of Statistical Learning: Data Mining, Inference, and Prediction](#), by Trevor Hastie, Robert Tibshirani, and Jerome Friedman (Chapter 2)
- [Foundations of Machine Learning](#), by Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. (Chapter 10)